CellWeave

Production Master System Design v1.0

Status: Authoritative Architecture Blueprint

Product Class: Mid-tier B2B scientific SaaS

Initial Market: US

Primary Customers: Small/mid-sized biotech companies and CRO/bioinformatics service teams

Company Size Target: ~10–200 employees

Scientific Scope: Downstream single-cell RNA-seq analysis

Regulatory Positioning: Research Use Only; non-diagnostic; PHI prohibited

Canonical Scientific Format: AnnData / `.h5ad`

---

1. Product Definition

CellWeave is a multi-tenant scientific analysis and collaboration platform for biotech and CRO teams that need to analyze processed single-cell RNA-seq datasets without maintaining fragmented R/Python pipelines.

CellWeave begins **after primary sequencing processing**.

CellWeave does not initially process:

```
FASTQ
↓
alignment
↓
read mapping
↓
UMI counting
```

Instead, CellWeave receives processed data such as:

```
10x matrices
.h5ad
CSV/TSV expression matrices
+
sample metadata
```

and provides:

```
Import
  ↓
```

```
Validation
  ↓
QC
  ↓
Filtering
  ↓
Normalization
  ↓
Feature selection
  ↓
Dimensionality reduction
  ↓
Clustering
  ↓
Cell annotation
  ↓
Marker discovery
  ↓
Differential expression
  ↓
Pathway analysis
  ↓
Experiment comparison
  ↓
Interactive results
  ↓
Collaboration
  ↓
Reproducible exports
```

The actual product is therefore not simply:

> "a website that runs Scanpy."

CellWeave's value is the combination of:

scientific computation + reproducibility + experiment organization + collaboration + provenance + usable visualization + operational infrastructure.

---

2. Core Product Promise

CellWeave should allow a scientist to go from:

> "I have processed single-cell data"

to:

> "My team has a reproducible, reviewable and comparable biological analysis"

without requiring that scientist to manually maintain Python/R environments.

The core promise is:

> **From processed single-cell data to reproducible biological insight, without maintaining custom analysis pipelines.**

---

3. Explicit V1 Scope

V1 shall support:

1. `.h5ad`
2. 10x filtered matrices
3. CSV/TSV expression matrix + metadata
4. Human datasets
5. Mouse datasets
6. Dataset validation
7. Per-sample QC
8. Doublet identification
9. User-controlled filtering
10. Normalization
11. Highly variable gene selection
12. PCA
13. Neighbor graph generation
14. UMAP
15. Leiden clustering
16. Automated cell-type annotation
17. Manual cell annotation correction
18. Marker-gene analysis
19. Differential-expression analysis
20. Pathway/enrichment analysis
21. Experiment comparisons
22. Interactive result exploration
23. Shared team projects
24. Comments

25. Analysis history
26. Immutable run provenance
27. Export bundles
28. Polar subscription + compute usage billing
29. Admin/operations console
30. Audit trails

---

4. Explicit Non-Goals

V1 shall **not** support:

```
raw FASTQ processing
clinical diagnosis
treatment recommendations
medical decision support
patient management
PHI workflows
native mobile apps
arbitrary user-written Python execution
arbitrary user-written R execution
n8n-style visual pipeline builder
real-time Google Docs-style editing
spatial transcriptomics
proteomics
ATAC-seq
full multi-omics
custom ML model training
```

These boundaries are deliberate.

They protect:

- scientific quality
- infrastructure costs
- security
- regulatory exposure
- engineering scope

---

5. Long-Term Product Expansion

CellWeave's intended evolution is:

```
scRNA-seq
   ↓
Advanced single-cell ML
   ↓
Spatial transcriptomics
   ↓
Multi-modal analysis
   ↓
Cross-study comparison
   ↓
Translational research workspace
```

V1 architecture shall therefore avoid assumptions that only RNA expression matrices will ever exist.

Core entities such as:

```
Dataset
Artifact
Experiment
AnalysisRun
Pipeline
WorkflowStep
```

must remain modality-neutral where practical.

---

6. Personas

Organization Owner

Usually founder, Head of Bioinformatics, CTO or operational owner.

Can:

- control subscription
- manage organization
- manage all users
- configure security
- configure retention
- manage billing
- access audit logs
- delete organization

Organization Admin

Can:

- manage members
- create projects
- manage project access
- see organization usage
- manage most organization settings

Cannot transfer ownership without Owner permission.

Bioinformatician

Primary analytical operator.

Can:

- import datasets
- configure metadata
- configure analysis parameters
- start/re-run analyses
- inspect QC
- annotate cells
- perform comparisons
- export scientific artifacts
- respond to scientists' requests

Scientist

Primary scientific consumer.

Can:

- explore analyses
- review QC
- interact with visualizations
- adjust approved parameters
- create comparisons
- annotate clusters
- comment
- export results

Cannot change organization administration.

Viewer

Read-only.

Can:

- view approved projects
- view analysis results
- download permitted reports

Cannot modify scientific state.

---

7. Authorization Model

Authorization must exist at two levels:

```
Organization
    ↓
Project
```

An organization role defines baseline capabilities.

Project permissions determine access to specific scientific projects.

Example:

```
Organization: BioNova

Alice – Admin
Bob   – Bioinformatician
Jane  – Scientist
Mark  – Viewer
```

But Bob may only have access to:

```
Project A
Project B
```

while Project C remains inaccessible.

Every protected domain object must ultimately resolve to an `organizationId`.

No endpoint shall trust an `organizationId` supplied by the browser without deriving and validating membership server-side.

---

8. Core User Journey

The primary workflow shall be:

```
Create Organization
        ↓
Create Project
        ↓
Upload Dataset
        ↓
Dataset Validation
        ↓
Configure Samples / Metadata
        ↓
Generate QC
        ↓
Scientist Reviews QC
        ↓
Confirm Filtering Parameters
        ↓
Start Analysis
        ↓
Pipeline Executes
        ↓
Review Clusters
        ↓
Automatic Annotation
        ↓
Human Review / Corrections
        ↓
Marker Analysis
        ↓
Condition Comparison
        ↓
Differential Expression
        ↓
Pathway Analysis
        ↓
Share / Discuss
        ↓
Export Reproducibility Bundle
```

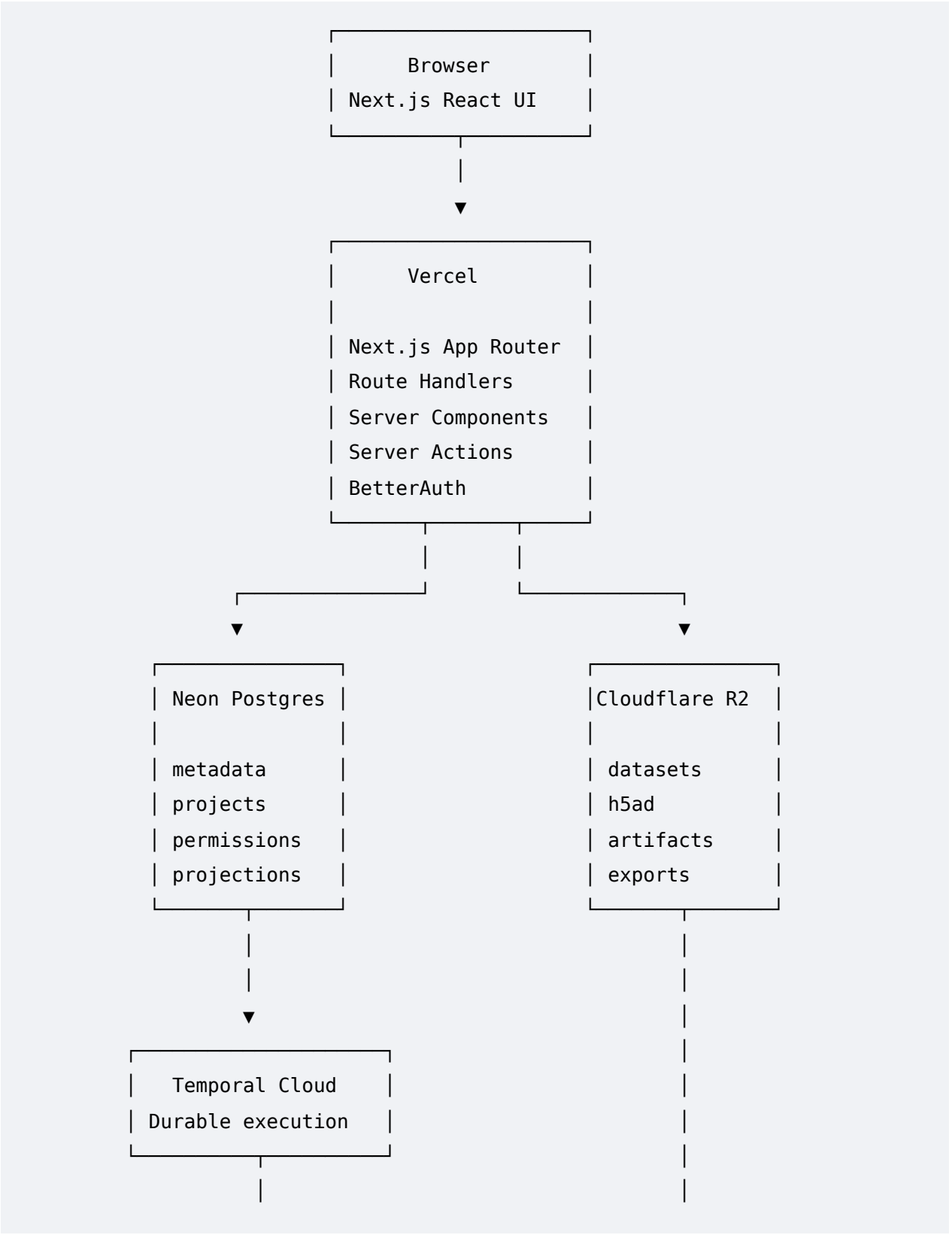
A key design rule:

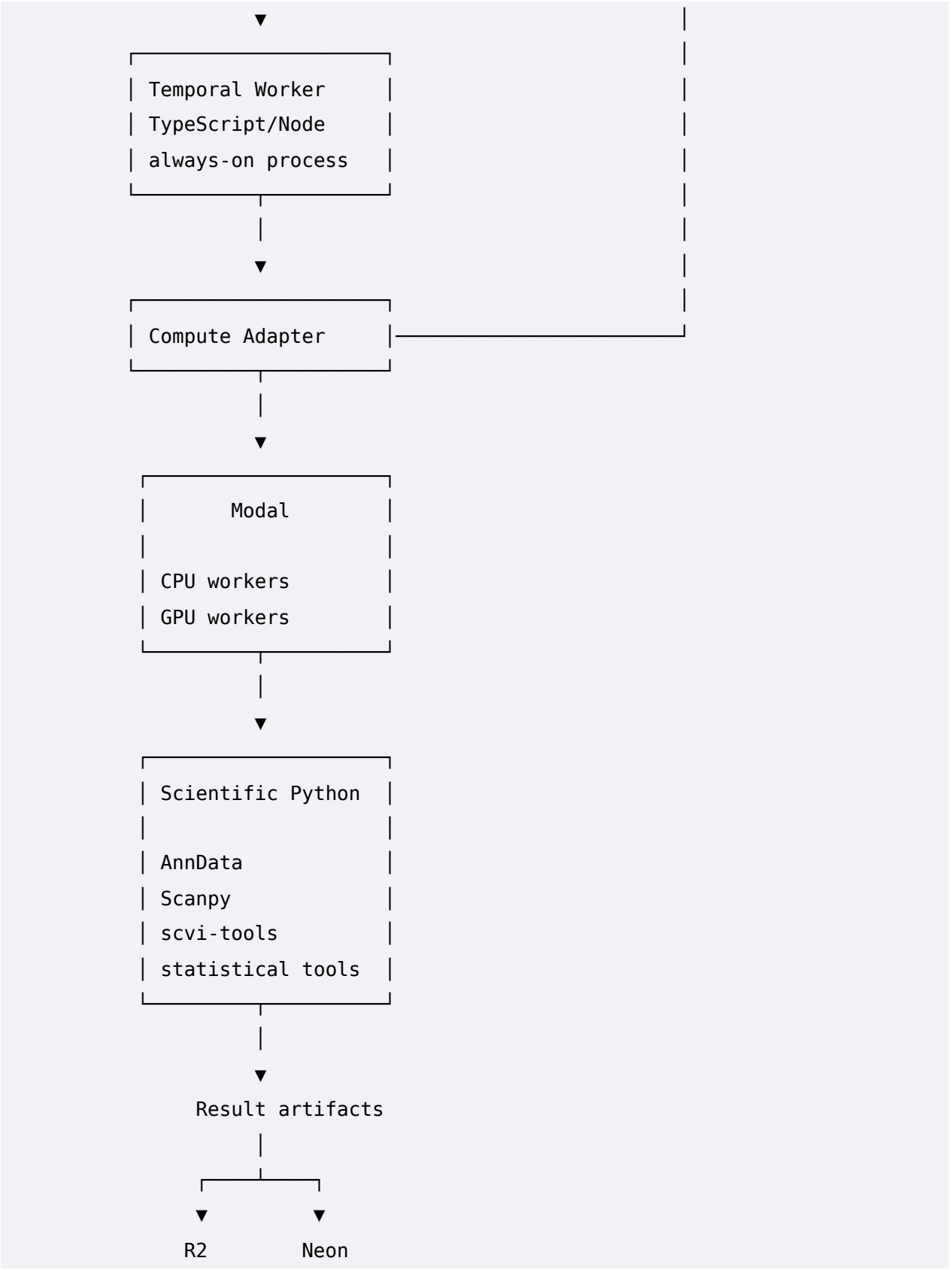
QC filtering should not silently destroy cells using universal thresholds.

Single-cell QC thresholds can differ between samples and biological contexts. CellWeave should calculate and recommend thresholds but expose them for review before downstream analysis.

---

9. High-Level Production Architecture





Temporal provides durable execution so workflow progress survives worker/process/network failures rather than relying on fragile request lifetimes.

Modal is used as elastic scientific compute rather than as the workflow source-of-truth. Its role is execution of CPU/GPU workloads and batch computation.

---

10. Repository Architecture

The project shall use separate repositories.

Repository 1 — `cellweave-web`

Contains:

```
Next.js
TypeScript
React
shadcn/ui
Motion
BetterAuth
Drizzle
Neon access
R2 integration
Polar integration
Temporal client
customer application
operations console
public API
webhooks
```

Repository 2 — `cellweave-orchestrator`

Long-running TypeScript process.

Contains:

```
Temporal workflows
Temporal activities
compute-provider interfaces
Modal adapter
workflow state projection
usage reporting
cancellation handling
retry policies
```

This repository exposes **no customer-facing REST API**.

It is an infrastructure worker.

Repository 3 — `cellweave-science`

Python scientific engine.

Contains:

```
cellweave_science/
    importers/
    validation/
    qc/
    normalization/
    embeddings/
    clustering/
    annotation/
    differential_expression/
    pathways/
    visualization/
    exports/
    provenance/

modal_app/
tests/
benchmarks/
reference_data/
```

Repository 4 — `cellweave-contracts`

Cross-language contracts.

Source of truth:

```
OpenAPI 3.1
JSON Schema
event schemas
run manifest schema
artifact manifest schema
```

Generates:

```
TypeScript types
Python Pydantic models
```

No duplicated hand-written payload interfaces across repositories.

Repository 5 — `cellweave-infra`

Infrastructure configuration.

Contains:

```
Terraform / OpenTofu
environment definitions
Cloudflare configuration
Temporal configuration
deployment definitions
monitoring configuration
security policies
CI templates
runbooks
```

---

11. Next.js Web Architecture

The web repository should follow feature/domain organization rather than placing business logic directly inside route files.

Recommended:

```
src/
├── app/
│   ├── (marketing)/
│   ├── (auth)/
│   ├── (app)/
│   │   ├── dashboard/
│   │   ├── projects/
│   │   ├── datasets/
│   │   ├── analyses/
│   │   ├── comparisons/
│   │   ├── billing/
│   │   └── settings/
│   ├── (ops)/
│   └── api/
│       ├── auth/
│       ├── v1/
│       ├── internal/
│       └── webhooks/
│           └── polar/
│
├── features/
│   ├── projects/
│   ├── datasets/
```

```
│   ├── analysis/
│   ├── comparison/
│   ├── collaboration/
│   ├── billing/
│   └── organization/
│
├── server/
│   ├── domain/
│   ├── application/
│   ├── infrastructure/
│   ├── authorization/
│   ├── database/
│   ├── storage/
│   └── integrations/
│
├── components/
│   ├── ui/
│   ├── scientific/
│   └── animations/
│
└── types/
```

Route Handlers should remain thin.

They should perform:

```
authentication
↓
validation
↓
authorization
↓
application service call
↓
response mapping
```

They must not contain scientific/business workflows.

---

12. API Design

All durable domain APIs use:

```
/api/v1/*
```

Examples:

```
POST   /api/v1/projects
GET    /api/v1/projects/:id

POST   /api/v1/uploads
POST   /api/v1/uploads/:id/complete

POST   /api/v1/datasets
GET    /api/v1/datasets/:id

POST   /api/v1/datasets/:id/validate

POST   /api/v1/analysis-runs
GET    /api/v1/analysis-runs/:id
POST   /api/v1/analysis-runs/:id/cancel

POST   /api/v1/comparisons
GET    /api/v1/comparisons/:id

PATCH  /api/v1/annotations/:id

POST   /api/v1/comments
GET    /api/v1/usage
```

Future public API can reuse these service boundaries.

Server Actions can be used for UI ergonomics but must call the same application layer.

---

13. Authentication

V1:

```
Email/password
Google OAuth
```

Better Auth supports Google authentication and organizations; the architecture shall also remain ready for OIDC/SAML enterprise SSO.

Enterprise future:

```
Okta
Microsoft Entra ID
custom OIDC
SAML 2.0
```

No authentication logic should be embedded in scientific services.

---

14. Dataset Upload Architecture

Large scientific files must **never** flow:

```
Browser
↓
Vercel function
↓
R2
```

Instead:

```
Browser
    │
    │ POST /uploads
    ▼
Next.js
    │
    │ authorizes upload
    │ returns signed upload data
    ▼
Browser ──────────► R2
```

Cloudflare R2 currently supports presigned operations and multipart uploads for large/resumable objects; multipart uploads can handle objects far beyond normal application upload sizes.

Upload lifecycle:

```
CREATED
↓
UPLOADING
↓
UPLOADED
↓
VERIFYING
↓
VERIFIED
↓
IMPORTED
```

Possible failure states:

```
FAILED
ABORTED
EXPIRED
QUARANTINED
```

---

15. Upload Integrity

R2 multipart ETags must not be treated as scientific content hashes.

After upload, an ingest worker calculates:

```
SHA-256
file size
detected format
compression metadata
```

The SHA-256 becomes the immutable input checksum recorded in provenance.

Duplicate files may then be detected within the same organization.

Cross-organization deduplication must never leak whether another customer has uploaded identical scientific data.

---

16. Upload Security

All uploads initially land in a logical quarantine state.

Validate:

```
maximum permitted size
declared format
actual binary/file signature
archive traversal
archive decompression ratio
file count
expected structure
schema sanity
dataset dimensions
```

Never run uploaded code.

ZIP extraction must reject:

```
../
absolute paths
symlinks outside sandbox
```

```
zip bombs
unexpected executables
```

---

17. Supported V1 Importers

H5AD importer

Input:

```
*.h5ad
```

Validates:

```
AnnData structure
X
obs
var
dimensions
unique identifiers
layers
species metadata
gene identifier format
```

AnnData is specifically structured around the data matrix plus observation, variable and other annotations, making it suitable as CellWeave's canonical internal representation.

10x importer

Support:

```
filtered_feature_bc_matrix.h5
```

and:

```
matrix.mtx.gz
features.tsv.gz
barcodes.tsv.gz
```

as a grouped import.

Generic matrix importer

Supports:

```
CSV
TSV
```

```
MTX
```

with mapping wizard.

User explicitly maps:

```
cells
genes
sample identifiers
conditions
batches
replicates
```

Ambiguous input must never be silently guessed.

---

18. Canonical Scientific Representation

After importing:

```
10x
CSV
TSV
h5ad
    ↓
Importer
    ↓
Canonical AnnData
```

Canonical file:

```
canonical.h5ad
```

The scientific pipeline operates against this canonical representation rather than maintaining vendor-specific downstream pipelines.

Future storage enum:

```
H5AD
ZARR
```

AnnData's ecosystem supports both `.h5ad` and Zarr representations, allowing the system to evolve toward chunked/cloud-native representations later if dataset scale requires it.

---

19. Scientific Pipeline V1

Stage 0 — Input validation

Validate:

```
dataset dimensions
sparse/dense representation
cell identifiers
gene identifiers
metadata
species
sample assignment
count availability
```

No analysis starts if the dataset is scientifically malformed.

Stage 1 — QC metrics

Generate per cell:

```
total counts
genes detected
mitochondrial percentage
ribosomal metrics
doublet score
```

QC must also be analyzed **per sample**, not only globally.

Stage 2 — Doublet detection

Default implementation:

```
Scrublet
```

Store:

```
doublet_score
predicted_doublet
algorithm version
parameters
```

Do not immediately delete predicted doublets.

Mark them first.

---

20. QC Review Gate

After QC:

```
QC_READY_FOR_REVIEW
```

The web application displays:

```
distributions
sample-level QC
mitochondrial percentages
gene counts
total counts
doublet predictions
suggested thresholds
```

The user then confirms filtering.

This produces an immutable:

```
QC Decision
```

containing:

```
thresholds
excluded cells
included cells
doublet handling
user
timestamp
reason
```

Only then proceeds downstream.

---

21. Normalization

Default validated workflow:

```
raw counts retained
↓
library-size normalization
↓
log1p
```

Never overwrite raw counts.

Canonical object should preserve raw counts in an appropriate layer.

---

22. Feature Selection

Default:

```
Highly Variable Genes
```

Parameters versioned.

For multi-sample datasets, feature selection should account for batch/sample information where appropriate.

---

23. Dimensionality Reduction

Default:

```
PCA
↓
neighbor graph
↓
UMAP
```

Store both:

```
numerical embeddings
visualization artifacts
```

The visualization must not be the only copy of derived coordinates.

---

24. Clustering

Default:

```
Leiden
```

User can select validated resolution options.

Every clustering change creates a new analysis artifact/version.

Never silently mutate historical cluster assignments.

Scanpy's current standard workflow similarly combines QC, normalization, variable-gene selection, PCA, neighbor graph, UMAP and Leiden clustering.

---

25. Cell-Type Annotation

Initial organisms:

```
Human
Mouse
```

Architecture:

```
AnnotationProvider
├── ReferenceModelProvider
├── ScANVIProvider
└── future providers
```

Automated annotation output must include:

```
predicted_label
confidence
annotation_method
model_name
model_version
reference_dataset
timestamp
```

If confidence is insufficient:

```
Unassigned
```

is preferable to fabricated certainty.

Human review is first-class.

A scientist/bioinformatician may:

```
accept
reject
replace
merge
split
```

cluster labels.

Manual corrections never erase automated provenance.

---

26. Advanced ML

scVI/scANVI belongs in advanced analysis rather than being required for every basic run.

GPU resources are used for workflows that benefit from them.

scvi-tools currently exposes models such as SCVI and SCANVI with support for batch-aware downstream analysis.

Typical routing:

```
basic QC
normalization
PCA
statistics
       ↓
CPU
```

versus:

```
scVI
scANVI
future foundation models
       ↓
GPU
```

---

27. Marker Gene Analysis

Cluster marker discovery may initially use validated non-parametric methods such as:

```
Wilcoxon
```

Results store:

```
gene
cluster
score
log fold change
p-value
adjusted p-value
method
parameters
```

---

28. Differential Expression

This area requires stricter scientific handling.

For:

```
cluster vs cluster marker discovery
```

cell-level analysis is acceptable when explicitly identified as marker discovery.

For:

```
Treatment vs Control
Drug A vs Drug B
```

```
Disease vs Healthy
```

CellWeave should default toward **sample-level/pseudobulk differential expression when biological replicates are available**.

Do not silently treat thousands of cells as thousands of independent biological replicates.

Scanpy's own documentation warns about this issue and specifically points toward pseudobulk approaches for more conservative sample-aware comparisons.

Comparison validation therefore checks:

```
number of biological samples
condition assignment
replicates
batch structure
cell counts
```

If comparison quality is weak, CellWeave displays a scientific warning.

---

29. Pathway Analysis

Architecture:

```
PathwayProvider
```

Rather than binding product logic to one database.

Every result records:

```
database
database version
gene universe
method
parameters
multiple-testing correction
```

External reference/pathway licensing must be reviewed before commercial production use.

No biological database should silently update underneath an existing analysis.

---

30. Experiment Model

Data hierarchy:

```
Organization
  ↓
Project
```

```
  ↓
Experiment
  ↓
Samples
  ↓
Dataset Versions
  ↓
Analysis Runs
```

Example:

```
Project:
Melanoma Study

Experiments:
├── Vehicle Control
├── Drug A
└── Drug B

Samples:
├── CTRL-01
├── CTRL-02
├── CTRL-03
├── DRUGA-01
├── DRUGA-02
└── DRUGA-03
```

This allows biologically valid comparison rather than comparing arbitrary files.

---

31. Experiment Comparison

Comparison entity:

```
comparisonId

projectId

groupA
groupB

cellPopulation

covariates
batchKeys
```

```
analysisMethod

createdBy
createdAt
```

Outputs can include:

```
cell population changes
marker changes
differential expression
pathway differences
QC differences
treatment response
```

Comparisons are immutable analytical objects.

Editing parameters creates a new comparison version.

---

32. Collaboration

Users can comment on:

```
project
dataset
analysis run
cluster
comparison
report
```

Example:

```
Scientist:
"Could Cluster 7 be CD8 effector T cells?"

Bioinformatician:
"Reviewed markers; relabeled."

System:
Cluster 7 annotation changed
NK Cell → CD8 Effector T Cell
```

That annotation event records:

```
before
after
```

```
actor
timestamp
reason
source
```

---

33. Reproducibility

Every completed run must create an immutable Run Manifest.

Example:

```
{
  "runId": "...",
  "pipeline": {
    "name": "cellweave-standard-scrna",
    "version": "1.4.0"
  },
  "container": {
    "digest": "sha256:..."
  },
  "input": {
    "datasetVersionId": "...",
    "sha256": "..."
  },
  "organism": "human",
  "parameters": {},
  "references": {},
  "software": {},
  "randomSeeds": {},
  "steps": [],
  "artifacts": []
}
```

Required provenance:

```
pipeline version
step versions
container digest
input checksum
parameters
random seeds
species
reference genome/build when available
```

```
annotation model/version
reference database versions
software dependency versions
outputs
timestamps
```

A pipeline version can never be edited after production use.

New behavior requires:

```
new pipeline version
```

---

34. Artifact Lineage

Every derived artifact knows:

```
what generated it
from which inputs
using which step
using which version
```

Example:

```
source.h5ad
   ↓
QC step
   ↓
qc.h5ad
   ↓
filter step
   ↓
filtered.h5ad
   ↓
normalize step
   ↓
normalized.h5ad
   ↓
cluster step
   ↓
clustered.h5ad
```

This enables selective recomputation.

---

35. No Unnecessary Recompute

Changing:

```
a chart color
```

must not rerun:

```
normalization
PCA
clustering
```

Changing:

```
cluster annotation
```

must not rerun UMAP.

Changing:

```
filter threshold
```

invalidates downstream artifacts and creates a new run branch.

Pipeline outputs should therefore form a DAG.

---

36. Temporal Workflow Architecture

Main workflow:

```
AnalysisWorkflow
```

Conceptually:

```
validate entitlement
↓
validate dataset
↓
prepare canonical input
↓
run QC
↓
WAIT FOR HUMAN QC CONFIRMATION
↓
filter
↓
normalize
↓
select features
```

```
↓
PCA
↓
neighbors
↓
UMAP
↓
clustering
↓
automatic annotation
↓
markers
↓
generate artifacts
↓
finalize provenance
↓
record usage
↓
complete
```

Temporal Signal:

```
confirmQC(parameters)
```

Temporal Signals also support:

```
cancel
manual annotation decision
resume after quota purchase
```

---

37. Temporal State vs Database State

Temporal is authoritative for:

```
workflow execution state
timers
retries
activity progress
```

Postgres is authoritative for:

```
business entities
customer-visible projections
```

```
permissions
billing state
scientific metadata
```

Workflow code itself must not perform arbitrary DB/network calls.

Non-deterministic work belongs in Temporal Activities.

---

38. Workflow IDs

Use stable deterministic IDs:

```
analysis-run:{analysisRunId}
```

This prevents accidental duplicate workflow execution.

Starting the same run twice must resolve to the same workflow identity rather than double-billing the user.

---

39. Run State Machine

```
DRAFT

UPLOADING

VALIDATING

QC_RUNNING

QC_REVIEW_REQUIRED

QUEUED

RUNNING

PAUSED_QUOTA

CANCEL_REQUESTED

CANCELLED

SUCCEEDED

FAILED
```

Terminal states:

```
SUCCEEDED
FAILED
CANCELLED
```

Failure does not automatically destroy intermediate outputs.

---

40. Workflow Step States

```
PENDING
QUEUED
RUNNING
SUCCEEDED
RETRYING
FAILED
SKIPPED
CANCELLED
```

Each attempt records:

```
attempt number
started time
completed time
compute resources
error classification
cost
logs reference
```

---

41. Retry Strategy

Retry:

```
network failures
temporary storage failures
Modal capacity failures
transient database failures
provider rate limiting
```

Do not retry indefinitely:

```
invalid dataset
unsupported schema
scientific validation failure
out-of-memory after resource ceiling
```

```
user permission failure
quota exhaustion
```

Failures must be classified.

---

42. Compute Architecture

CellWeave's science engine exposes logical jobs:

```
validate_dataset()
run_qc()
filter_cells()
normalize()
select_hvg()
run_pca()
build_neighbors()
run_umap()
cluster()
annotate()
find_markers()
run_differential_expression()
run_pathway_analysis()
generate_exports()
```

Temporal does not execute these locally.

It delegates them through:

```
ComputeProvider
```

---

43. Compute Provider Abstraction

TypeScript contract:

```
ComputeProvider

submit()
status()
cancel()
result()
```

Initial implementation:

```
ModalComputeProvider
```

Future:

```
RunPodComputeProvider
AWSBatchComputeProvider
DedicatedComputeProvider
```

No domain logic may directly call Modal throughout the application.

That prevents compute-provider lock-in.

---

44. Resource Profiles

Scientific activities declare resource class:

```
CPU_SMALL

CPU_MEMORY

CPU_HIGH_MEMORY

GPU_SMALL

GPU_LARGE
```

The scientific engine maps these logical profiles to actual Modal resources.

This allows hardware mapping to change without changing pipeline definitions.

---

45. Resource Estimation

Before expensive execution, estimate:

```
cells
genes
matrix sparsity
dataset size
requested method
batch count
```

Then determine:

```
memory requirement
CPU class
GPU requirement
expected compute units
```

Jobs exceeding safe bounds are rejected or require a larger plan.

This prevents accidental $500 compute jobs.

---

46. Cost Guardrails

Every analysis receives:

```
estimatedComputeUnits
maxComputeUnits
maxRuntime
maxMemoryClass
```

Compute must terminate if an enforced ceiling is reached.

Organization controls may optionally include:

```
monthly compute budget
per-run budget
concurrent-job limit
```

---

47. Storage Architecture

R2 is private.

No dataset bucket is public.

Object path convention:

```
org/{orgId}/
  project/{projectId}/
    datasets/{datasetVersionId}/
      source/
      canonical/
    runs/{runId}/
      steps/{stepKey}/
      artifacts/
      visualizations/
      exports/
```

Example:

```
org/abc/
project/proj-1/
runs/run-123/
```

```
steps/umap/
embedding.arrow
```

---

48. Object Access

Downloads use short-lived signed URLs.

Authorization flow:

```
Browser
↓
CellWeave API
↓
permission check
↓
generate signed URL
↓
Browser downloads from R2
```

Never expose permanent public object URLs.

---

49. Database Architecture

Use:

```
PostgreSQL
Neon
Drizzle ORM
```

Postgres stores **metadata**, not large scientific matrices.

R2 stores heavy binary artifacts.

---

50. Core Database Entities

Core tables:

```
organizations
organization_settings

projects
project_members

experiments
samples
```

```
datasets
dataset_versions
dataset_files
upload_sessions
imports

pipeline_definitions
pipeline_versions

analysis_runs
analysis_run_parameters
workflow_steps

artifacts
artifact_lineage

cluster_annotations
annotation_revisions

comparisons
comparison_versions

comments

reports

audit_events

notifications

usage_ledger
usage_aggregates

billing_accounts
billing_subscriptions
entitlements

polar_events
polar_sync_outbox

idempotency_keys
```

```
service_events

data_deletion_jobs
```

Better Auth owns its own authentication/session/organization-related tables where appropriate.

Do not duplicate Better Auth records without a domain reason.

---

51. Primary Keys

Use sortable server-generated identifiers such as:

```
UUIDv7
```

Never expose sequential integer IDs externally.

---

52. `projects`

Important fields:

```
id
organizationId

name
description

status

createdBy
createdAt
updatedAt

archivedAt
```

Indexes:

```
organizationId
organizationId + status
createdAt
```

---

53. `experiments`

```
id
organizationId
projectId
```

```
name
description

organism
tissue
diseaseContext

createdBy
createdAt
```

Do not force every scientific concept into nullable experiment columns.

Additional structured metadata can use validated JSON schemas.

---

54. `samples`

```
id
organizationId
experimentId

sampleKey
displayName

condition
batch
biologicalReplicate
technicalReplicate

metadataJson
```

`sampleKey` unique within experiment.

---

55. `datasets`

Logical dataset:

```
id
organizationId
projectId

name
description
status
```

```
createdBy
createdAt
```

---

56. `dataset_versions`

Immutable dataset revision:

```
id
organizationId
datasetId

versionNumber

sourceFormat
canonicalFormat

organism
geneIdScheme
referenceBuild

cellCount
geneCount

sourceChecksum
canonicalChecksum

createdAt
```

Never overwrite a dataset version after it becomes input to an analysis.

---

57. `analysis_runs`

```
id
organizationId
projectId
datasetVersionId

pipelineVersionId

status

createdBy
```

```
estimatedComputeUnits
actualComputeUnits

queuedAt
startedAt
completedAt

failureCode
failureSummary

temporalWorkflowId
```

---

58. `workflow_steps`

```
id
organizationId
analysisRunId

stepKey
stepVersion

status
attempt

computeProfile

startedAt
completedAt

inputManifest
outputManifest

errorCode
```

---

59. `artifacts`

```
id
organizationId
analysisRunId
workflowStepId
```

```
  artifactType

  storageProvider
  bucket
  objectKey

  sha256
  sizeBytes
  mimeType

  createdAt
```

Artifact types:

```
  SOURCE_DATASET
  CANONICAL_DATASET
  QC_REPORT
  NORMALIZED_DATASET
  EMBEDDING
  CLUSTERS
  ANNOTATIONS
  MARKER_TABLE
  DE_TABLE
  PATHWAY_TABLE
  FIGURE
  REPORT
  RUN_MANIFEST
  EXPORT_BUNDLE
```

---

60. Artifact Lineage

`artifact_lineage`:

```
  parentArtifactId
  childArtifactId
  relationship
```

Allows:

```
  source → normalized → clustered → annotated
```

to be reconstructed.

---

61. Annotation Revisions

Never mutate annotations without history.

Tables:

```
cluster_annotations
annotation_revisions
```

Revision:

```
clusterId
previousLabel
newLabel
confidence
method
reason
changedBy
createdAt
```

---

62. Audit Events

Audit events must be append-only at application level.

Examples:

```
user.invited
member.role_changed

dataset.uploaded
dataset.deleted

analysis.started
analysis.cancelled

annotation.changed

export.downloaded

billing.plan_changed

organization.retention_changed
```

Fields:

```
id
organizationId
actorType
actorId

action

resourceType
resourceId

metadata

ipHash / request metadata where appropriate

createdAt
```

Never place raw scientific datasets in audit metadata.

---

63. Tenant Isolation

Every tenant-owned domain table contains:

```
organizationId
```

Every repository query accepts:

```
TenantContext
```

instead of arbitrary tenant IDs.

Example conceptually:

```
projectRepository.getById({
  tenant,
  projectId
})
```

not:

```
db.query.project.findFirst({
  id: projectId
})
```

Authorization integration tests must attempt cross-tenant access for every resource type.

---

64. Defense-in-Depth Database Policy

Where practical, production domain tables should additionally use Postgres Row Level Security.

Application filtering remains mandatory.

RLS is additional protection, not replacement for authorization logic.

Background service credentials should have separate explicit privileges.

Science containers should **not receive primary database credentials**.

---

65. Scientific Worker Permissions

Scientific compute receives only:

```
specific input signed URL
specific output upload capability
run manifest fragment
required parameters
```

It does not receive:

```
BetterAuth credentials
Polar credentials
full R2 master credentials
database admin credentials
```

This significantly limits blast radius.

---

66. Internal Service Authentication

Service-to-service communication uses short-lived signed service credentials.

Examples:

```
orchestrator → Next.js internal API
Next.js → Temporal
```

Each service identity has:

```
issuer
audience
scope
expiration
```

Static shared master API keys should not become the permanent architecture.

---

67. Billing Architecture

Payment provider:

Polar

CellWeave shall use:

```
subscription
+
included credits
+
metered usage
```

Polar currently supports recurring subscriptions, usage meters and prepaid credits, which aligns well with CellWeave's compute/storage economics.

---

68. Billing Source of Truth

Important:

```
CellWeave usage ledger
        ↓
Polar
```

not:

```
Polar
↓
CellWeave has no internal record
```

The internal immutable usage ledger remains the source for:

```
entitlement enforcement
internal analytics
cost reconciliation
support investigations
```

Polar is the commercial billing system.

---

69. Usage Units

Do not expose raw infrastructure concepts directly unless useful.

Internal measurements:

```
cpu_seconds
gpu_seconds
memory_class
bytes_processed
storage_byte_hours
egress_bytes
```

Customer-facing:

```
Compute Credits
Storage Allowance
```

The conversion between raw usage and credits is configuration-driven.

---

70. Polar Usage Flow

```
Scientific step completes
        ↓
usage event created
        ↓
usage_ledger INSERT
        ↓
billing outbox INSERT
        ↓
Polar event sent
        ↓
Polar acknowledgment stored
```

Event idempotency key:

```
runId:stepId:attempt
```

Duplicate retries may never duplicate customer charges.

---

71. Entitlements

CellWeave maintains local entitlements:

```
maxUsers
maxProjects
storageBytes
includedComputeCredits
concurrentRuns
```

```
retentionDays
advancedML
SSO
auditExport
priorityCompute
```

Polar webhook updates subscription projection.

Application logic then derives entitlements.

Polar itself does not decide whether an analysis may run.

---

72. Billing Plans

Architecture should support something like:

```
Team
Growth
Enterprise
```

without hardcoding plan names into domain logic.

Use:

```
PlanDefinition
Entitlement
```

Plan changes become configuration rather than conditionals scattered throughout source code.

---

73. Polar Webhooks

Route:

```
POST /api/webhooks/polar
```

Requirements:

```
signature verification
unique external event ID
idempotent processing
event persistence
retry-safe handlers
dead-letter visibility
```

Webhook receipt must be persisted before business processing.

---

74. Visualization Architecture

The browser must never download an entire multi-gigabyte `.h5ad` simply to display UMAP.

Science pipeline generates optimized visualization artifacts.

Example:

```
coordinates
cluster index
sample index
selected metadata columns
```

For smaller datasets:

```
Canvas
```

For larger datasets:

```
WebGL
```

Recommended rendering abstraction:

```
ScientificScatterRenderer
```

which can switch implementation according to dataset size.

---

75. Results UI

Analysis result page:

```
Overview

QC

UMAP

Clusters

Cell Types

Markers

Differential Expression

Pathways

Comparisons
```

```
Artifacts

Provenance

Activity
```

Every chart should answer:

```
What am I seeing?
Which run produced it?
Which parameters were used?
Can I export the underlying values?
```

---

76. Dashboard

Organization dashboard focuses on operational scientific work:

```
Active projects
Recent analyses
Failed analyses
Analyses awaiting QC review
Compute usage
Storage usage
Team activity
```

Do not turn the dashboard into decorative analytics.

---

77. Project Page

```
Project Overview
Experiments
Samples
Datasets
Analysis Runs
Comparisons
Comments
Exports
Activity
```

---

78. Analysis Creation UX

Wizard:

```
1. Select dataset

2. Review dataset metadata

3. Select pipeline

4. Configure validated parameters

5. Estimate compute usage

6. Start
```

Before expensive execution:

```
Estimated usage:
≈ X Compute Credits
```

No unexpected compute.

---

79. Error UX

Scientific errors must not show:

```
500 Internal Server Error
```

as the primary explanation.

Domain-aware examples:

```
GENE_IDS_UNRECOGNIZED

SAMPLE_METADATA_MISSING

INSUFFICIENT_REPLICATES

DATASET_TOO_LARGE

OUT_OF_MEMORY

ANNOTATION_MODEL_UNAVAILABLE

UNSUPPORTED_SPECIES

QUOTA_EXCEEDED
```

Each has:

```
user-facing message
technical details
recommended action
support correlation ID
```

---

80. Notification System

V1:

```
In-app
Email via Resend
```

Notify on:

```
analysis complete
analysis failed
QC review required
comparison complete
invitation
billing issue
storage threshold
compute threshold
```

Never email scientific raw results unless intentionally exported.

---

81. Internal Operations Console

Implemented inside:

```
/(ops)
```

but protected by completely separate staff authorization.

Capabilities:

```
tenant lookup
subscription state
usage
storage
running analyses
failed workflows
Temporal workflow ID
Modal job IDs
retry/cancel
```

```
billing sync status
webhook failures
notifications
system incidents
feature flags
```

---

82. Customer Data Access by Staff

Default:

```
staff cannot inspect customer scientific data
```

Break-glass access requires:

```
privileged role
explicit reason
short expiration
audit event
```

No silent customer impersonation.

---

83. Scientific Provenance UI

Users should be able to open:

```
Run → Provenance
```

and see:

```
Input checksum

Pipeline 1.4.0

Scanpy version
AnnData version
Annotation model
Reference resource version

QC thresholds

Clustering resolution

Differential expression method

Container image digest
```

```
Run timestamps
```

This is a product feature, not hidden engineering metadata.

---

84. Exports

Users can export:

```
processed .h5ad
CSV tables
figures
QC report
DE results
pathway results
analysis report
```

and:

Reproducibility Bundle

Containing:

```
run-manifest.json
parameters.json
software-versions.json
artifact-manifest.json
results/
README
```

The platform should reduce lock-in anxiety.

---

85. Data Retention

Organization configuration:

```
source file retention
derived artifact retention
failed run retention
export retention
```

Plans determine allowable limits.

Deletion lifecycle:

```
ACTIVE
↓
SOFT_DELETED
↓
RECOVERY_WINDOW
↓
PURGE_PENDING
↓
PURGED
```

Deletion must cascade through artifacts without accidentally touching resources belonging to another tenant.

---

86. Organization Deletion

Organization deletion requires:

```
Owner authentication
confirmation
optional export
grace period
scheduled purge
```

Purge includes:

```
projects
datasets
artifacts
comments
scientific results
API credentials
```

Financial/audit records are retained only where legitimately required and minimized.

---

87. Security Baseline

CellWeave shall implement:

```
TLS everywhere

encryption at rest

private object storage
```

```
short-lived signed URLs

least-privilege credentials

strict tenant authorization

rate limiting

secure headers

CSRF protection where relevant

SameSite secure cookies

session rotation

webhook verification

idempotency

dependency scanning

secret scanning

container vulnerability scanning

audit logging
```

---

88. Research-Use-Only Boundary

Product UI and Terms must state:

```
Research Use Only
Not for diagnostic use
Not intended to make clinical decisions
```

CellWeave V1 shall prohibit uploading identifiable patient information/PHI.

The product can still handle legitimate de-identified research datasets subject to customer obligations and legal review.

---

89. SOC 2 Readiness

Do not pursue certification before the business warrants it.

But architecture/processes begin SOC 2-friendly.

Controls include:

```
individual staff accounts
MFA
production access review
least privilege
change approval
CI/CD evidence
audit logging
incident response
backup testing
vulnerability management
vendor inventory
secret management
employee offboarding
security policies
```

---

90. Secrets

Never place secrets in:

```
Git
browser bundle
scientific artifacts
run manifests
logs
```

Separate secrets per environment.

Examples:

```
BetterAuth
Google OAuth
Neon
R2
Temporal
Modal
Polar
Resend
Sentry
```

Rotate credentials independently.

---

91. Observability

Three pillars:

```
Logs
Metrics
Traces
```

Recommended:

```
OpenTelemetry
Sentry
central structured logs
Temporal observability
Modal metrics
```

Every request/run should carry:

```
requestId
organizationId
analysisRunId
workflowId
```

where security appropriate.

Do not log scientific matrices.

---

92. Important Metrics

Application:

```
HTTP latency
HTTP error rate
auth failures
DB latency
R2 failures
```

Workflow:

```
queued runs
running runs
failed runs
step retry rate
time per step
cancellation rate
```

Scientific compute:

```
CPU seconds
GPU seconds
memory
OOM events
cells processed
cost per run
```

Business:

```
active organizations
runs/org
compute margin
storage/org
upgrade events
```

---

93. SLO

Initial production target:

```
Web/API monthly availability:
99.9%
```

Additional internal objectives:

```
workflow request durability
>99.99%

successful completed upload verification
>99.5%

billing event reconciliation
100%

cross-tenant access
0
```

Scientific validation errors are not counted as infrastructure failures.

---

94. Disaster Recovery

Metadata:

```
Neon PITR
+
scheduled logical backups
```

Neon provides point-in-time restore capabilities, which should be tested as part of the DR process rather than merely assumed to work.

Initial targets:

```
Metadata RPO ≤ 1 hour
Metadata RTO ≤ 4 hours
```

R2 scientific objects:

```
soft deletion
delayed purge
checksums
artifact manifests
```

For higher enterprise requirements, add cross-provider encrypted replication.

---

95. Backup Testing

Quarterly:

```
restore DB into isolated environment
verify schema
verify project metadata
resolve R2 references
open representative artifacts
run checksum validation
```

A backup that has never been restored is not considered proven.

---

96. Environments

Required:

```
local
development
staging
production
```

plus:

```
Vercel previews
```

Production customer data must never be copied into preview deployments.

Development/test datasets use public/synthetic data.

---

97. Environment Isolation

Separate:

```
Neon projects/branches
R2 prefixes or buckets
Temporal namespaces
Modal environments
Polar sandbox/production
OAuth credentials
secrets
```

Production credentials are never shared with development.

---

98. Branch/Preview Database Strategy

Preview builds may use ephemeral Neon branches generated from:

```
sanitized staging schema/data
```

Never branch directly from production customer scientific metadata for arbitrary pull requests.

---

99. CI/CD — Web

Pull request:

```
install
↓
lint
↓
format
↓
TypeScript typecheck
↓
unit tests
↓
authorization tests
↓
```

```
DB migration validation
↓
build
↓
Playwright smoke
↓
preview
```

Production merge:

```
migration safety gate
↓
deploy
↓
health check
↓
synthetic smoke test
↓
progressive feature activation
```

---

100. CI/CD — Scientific Engine

Every change:

```
Python lint
↓
type checks
↓
unit tests
↓
scientific regression suite
↓
golden dataset tests
↓
container build
↓
SBOM
↓
vulnerability scan
↓
benchmark
```

A scientific algorithm update cannot deploy merely because unit tests pass.

---

101. Scientific Regression Testing

Maintain public benchmark datasets.

Tests evaluate:

```
cell counts after filtering
QC distributions
PCA properties
cluster consistency
known marker recovery
annotation accuracy
DE direction
pathway expectations
runtime
memory
```

Do not require byte-identical UMAP coordinates where stochastic algorithms make this inappropriate.

Use scientifically meaningful tolerances.

---

102. Golden Dataset Suite

At least:

```
small smoke dataset
human PBMC dataset
mouse dataset
multi-sample dataset
batch-effect dataset
known-treatment dataset
large benchmark dataset
```

Every production pipeline version must pass these benchmarks.

---

103. Pipeline Release Strategy

Pipeline versions are immutable.

Example:

```
1.2.0
1.3.0
1.4.0
```

Existing analysis:

```
remains pinned to 1.3.0
```

If 1.4.0 becomes default:

```
new runs → 1.4.0
old run provenance → 1.3.0
```

A user may explicitly choose:

```
Re-run with latest pipeline
```

---

104. Database Migration Strategy

Use:

```
expand
↓
migrate/backfill
↓
switch application
↓
contract
```

Avoid production migrations that:

```
lock huge tables
drop used columns immediately
rewrite large tables synchronously
```

Backward compatibility must exist during deployment windows.

---

105. API Compatibility

Contracts are versioned.

Breaking public/API changes require:

```
/v2
```

Internal contracts use explicit schema versions.

Scientific job payload:

```
schemaVersion
pipelineVersion
```

must always be present.

---

106. Idempotency

Required for:

```
upload completion
run creation
workflow startup
Polar webhook handling
billing events
notification sending
artifact registration
```

Client write requests may accept:

```
Idempotency-Key
```

Temporal workflow IDs provide additional protection for analysis execution.

---

107. Run Creation Consistency

Potential failure:

```
DB row created
↓
Temporal start fails
```

Solution:

```
analysis_run.status = START_PENDING
```

Then:

```
start Temporal workflow using deterministic ID
```

If start fails:

```
reconciliation process retries
```

Starting the same workflow ID remains idempotent.

No paid job is silently lost.

---

108. Background Reconciliation

Periodic reconciliation checks:

```
START_PENDING runs

Temporal workflow projections

stale uploads

orphaned multipart uploads

unreported usage

unsynced Polar events

stuck deletion jobs

expired signed-upload sessions
```

This is critical production plumbing.

---

109. Next.js Performance Strategy

Use Server Components for data-heavy initial views.

Use client components only for:

```
interactive plots
forms
filters
animations
live run status
```

Use TanStack Query for dynamic client data.

Avoid turning the entire dashboard into a client-rendered SPA.

---

110. Animation Architecture

Keep animations standardized:

```
components/
└── animations/
    ├── ModalAnimations.ts
    ├── FormAnimations.ts
    ├── PageAnimations.ts
    └── ResultAnimations.ts
```

Motion must improve perceived responsiveness without slowing scientific navigation.

Respect:

```
prefers-reduced-motion
```

---

111. Responsive Design

Primary usage is desktop.

However:

```
laptop
tablet
mobile review
```

must remain functional.

Mobile does not need full high-density scientific analysis parity.

It should support:

```
run status
comments
basic charts
notifications
billing
project review
```

Complex UMAP manipulation may provide a reduced interaction mode on small screens.

---

112. Accessibility

Target:

```
WCAG 2.2 AA
```

Scientific plots should not communicate meaning by color alone.

Provide:

```
legends
labels
tables
downloadable underlying values
keyboard navigation
```

---

113. Abuse / Cost Protection

Attack:

```
user submits thousands of huge jobs
```

Mitigation:

```
plan quotas
per-org concurrency
rate limiting
compute admission
monthly budget
workflow priority
```

---

114. Threat Model

Major threats:

Threat	Primary Mitigation
Cross-tenant IDOR	TenantContext + authorization + tests + optional RLS
Public scientific files	Private R2 + short-lived signed URLs
Malicious archive	sandboxed extraction + limits
Worker compromis	scoped URLs; no

e	DB credentials
Duplicate workflow	deterministic Temporal workflow IDs
Duplicate billing	immutable usage IDs
Webhook replay	signature + external-event uniqueness
Privilege escalation	explicit RBAC
Compute abuse	quotas + resource ceilings
Secret leak	isolated secret stores
Dependency compromise	lockfiles + SBOM + scanning
Scientific silent regression	benchmark/golden dataset gates

---

115. Scientific Failure vs Platform Failure

Separate:

```
PLATFORM_ERROR
```

from:

```
SCIENTIFIC_VALIDATION_ERROR
```

Example scientific validation:

```
only one biological replicate exists
```

This is not a platform outage.

Example platform failure:

```
Modal unavailable
```

This is infrastructure.

Observability and support must distinguish them.

---

116. Customer-Facing Scientific Warnings

Examples:

```
Low number of biological replicates

Strong batch confounding detected

High mitochondrial content

Large doublet fraction

Cell annotation confidence low

Comparison contains insufficient cells

Reference organism mismatch
```

Warnings remain part of run provenance.

---

117. Data Governance

Organization settings include:

```
default retention
download permissions
export permissions
project visibility
SSO policy later
MFA requirements later
```

Enterprise future:

```
customer-managed storage
dedicated database
dedicated compute
regional isolation
```

The base architecture must not require those modes for ordinary customers.

---

118. Feature Flags

Feature flags scoped by:

```
environment
organization
user
```

Examples:

```
advanced_scvi

new_annotation_model

new_umap_renderer

seurat_import

spatial_beta
```

Scientific feature rollouts should begin with internal/test tenants before broad availability.

---

119. Seurat Support — V1.5/V2

Future importer:

```
Seurat object
↓
isolated R environment/converter
```

```
↓
canonical AnnData
```

R conversion must remain isolated from the core Python environment.

Do not pollute the standard science worker with every R dependency.

---

120. Parse Biosciences Support

Future vendor-specific import adapter.

Architecture:

```
ParseImporter
implements
DatasetImporter
```

Downstream analysis remains unchanged.

---

121. Importer Interface

Conceptually:

```
DatasetImporter

detect()
validate()
inspect()
convert()
metadata()
```

Implementations:

```
H5ADImporter

TenXH5Importer

TenXMtxImporter

MatrixImporter

future SeuratImporter

future ParseImporter
```

---

122. Storage Interface

```
ObjectStorage

createUpload()
completeUpload()
createDownloadUrl()
headObject()
deleteObject()
copyObject()
```

Initial:

```
R2ObjectStorage
```

Future:

```
S3ObjectStorage
```

No business service imports Cloudflare-specific code directly.

---

123. Billing Interface

```
BillingProvider

createCheckout()
getSubscription()
reportUsage()
customerPortal()
```

Initial:

```
PolarBillingProvider
```

Again, domain code should not know about Polar-specific payload structures.

---

124. Notification Interface

```
NotificationProvider
```

Implementations:

```
InAppNotificationProvider
EmailNotificationProvider
```

Future:

```
SlackNotificationProvider
TeamsNotificationProvider
```

---

125. Public API Future

V1 customers use UI.

But internal APIs already follow stable contracts.

Later API:

```
POST /api/v1/datasets
POST /api/v1/analysis-runs
GET  /api/v1/analysis-runs/:id
GET  /api/v1/artifacts/:id
```

Authentication:

```
organization-scoped API tokens
```

API token scopes:

```
dataset:read
dataset:write
analysis:read
analysis:write
artifact:read
```

---

126. Production Operational Runbooks

Required before launch:

```
Database outage

R2 outage

Temporal outage

Modal outage

Polar outage

OAuth outage

Run stuck
```

```
Billing mismatch

Customer data deletion

Credential compromise

Cross-tenant access incident

Pipeline scientific regression

Bad deployment rollback
```

Each runbook defines:

```
detection
severity
owner
containment
recovery
customer communication
postmortem
```

---

127. Incident Severity

```
SEV-1
cross-tenant exposure
major data loss
security compromise

SEV-2
major analysis outage
billing corruption

SEV-3
degraded feature
individual workflow failure pattern

SEV-4
minor defect
```

SEV-1 automatically triggers security incident procedure.

---

128. Support Diagnostics

Every error shown to user gets:

```
Correlation ID
```

Support can search:

```
request logs
analysis run
Temporal workflow
Modal job
audit history
```

without accessing scientific raw content by default.

---

129. Admin Metrics

Operations dashboard:

```
Queued analyses
Running analyses
Failed analyses

CPU consumption
GPU consumption

Compute cost
Polar revenue
gross compute margin

R2 storage

Neon DB load

Temporal failures

Webhook failures
```

---

130. Scalability Model — Stage 1

Early production:

```
Vercel
Neon
R2
```

```
Temporal Cloud
1–2 orchestrator replicas
Modal autoscaling
```

No Kubernetes.

---

131. Scalability Model — Stage 2

As customers increase:

```
multiple Temporal Worker replicas

task queues split:
standard
high-memory
gpu
enterprise

database read scaling

result caching

higher Modal concurrency

per-org workflow limits
```

---

132. Scalability Model — Stage 3

At substantial enterprise scale:

```
regional compute pools
dedicated enterprise task queues
optional AWS Batch/ECS
dedicated customer storage
dedicated customer DB
cross-region DR
```

The product should reach this point before Kubernetes becomes a default requirement.

---

133. Why This Architecture Scales

Large data does not flow through application servers.

```
Browser → R2
```

Scientific computation does not consume Vercel resources.

```
Temporal → Modal
```

Long-running work does not depend on HTTP request lifetimes.

```
Temporal Durable Workflow
```

Large outputs do not live in Postgres.

```
R2
```

Postgres primarily handles structured metadata.

The four heaviest concerns therefore scale independently:

```
web
database
storage
compute
```

---

134. Cost Model

Track cost per:

```
Organization
Project
Run
Workflow Step
```

Internal cost record:

```
cpu cost
gpu cost
storage cost
egress cost
third-party cost
```

This is necessary to know:

```
customer revenue
-
cost to serve
=
gross margin
```

Polar currently also supports usage/cost-oriented billing primitives, but CellWeave still maintains its own operational ledger.

---

135. Scientific Compute Caching

Cache only when identity is safe.

Cache key may include:

```
input checksum
pipeline version
step version
parameters hash
reference version
```

Never return another tenant's artifact simply because hashes match.

Shared computational caching across customers is prohibited initially.

Tenant-scoped cache is safe.

---

136. Cancellation

User presses:

```
Cancel Analysis
```

Flow:

```
Next.js
↓
Temporal cancellation request
↓
current Modal compute cancelled if possible
↓
remaining steps skipped
↓
usage finalized
↓
run = CANCELLED
```

Previously created artifacts can remain temporarily for debugging/retention rules.

---

137. Quota Pause

If credits become insufficient before a high-cost stage:

```
RUNNING
↓
PAUSED_QUOTA
```

The workflow remains durable.

User:

```
tops up / upgrades
```

Then Temporal receives:

```
resume signal
```

and continues.

---

138. Polar Customer Mapping

Database stores:

```
organizationId
polarCustomerId
polarSubscriptionId
```

Billing is organization-scoped.

Individual scientist accounts do not own subscriptions.

---

139. Email Strategy

Resend for application email.

Templates:

```
invite
analysis complete
analysis failed
QC review needed
billing problem
subscription changed
usage threshold
```

All emails include organization context.

---

140. Product Analytics

Track product events without scientific content.

Safe events:

```
project_created
dataset_import_started
analysis_started
analysis_completed
comparison_created
export_created
upgrade_clicked
```

Never send:

```
gene names
patient identifiers
scientific matrices
biological conclusions
```

to ordinary product analytics.

---

141. Privacy Boundary

Third-party analytics receive only minimum operational metadata.

Scientific data should flow only through providers necessary to run CellWeave.

Maintain a documented subprocessor inventory.

---

142. AI Interpretation — V2

Not part of production V1 scientific truth path.

Later architecture:

```
Structured scientific results
↓
Evidence retrieval
↓
Interpretation engine
↓
Cited draft
↓
Scientist review
```

The LLM may explain.

It may not replace:

```
statistics
cell annotation algorithms
DE calculations
pathway calculations
```

Every statement must point back to structured results and sources.

---

143. Data Model for Future AI

Store structured observations now:

```
cell population deltas
marker tables
DE tables
pathway tables
QC metrics
```

Do not store only screenshots.

This makes future evidence-based interpretation possible.

---

144. Code Quality Rules — TypeScript

Mandatory:

```
strict TypeScript
no any
no unsafe casts without reason
Zod/request validation
explicit domain types
discriminated unions for states
exhaustive switches
```

Domain IDs may use branded types:

```
OrganizationId
ProjectId
AnalysisRunId
DatasetVersionId
```

to reduce accidental identifier mixing.

---

145. Python Quality Rules

Mandatory:

```
type hints
Pydantic schemas
Ruff
mypy/pyright where practical
pytest
dependency pinning
deterministic seeds
structured logging
```

Scientific functions should be pure where practical.

---

146. Logging Rules

Structured JSON logs.

Never use:

```
print(adata)
```

on customer datasets.

Never log:

```
matrix contents
gene-expression vectors
full filenames containing sensitive metadata
```

Prefer identifiers.

---

147. Local Development

Developer environment can run:

```
Next.js
local Postgres or dev Neon
Temporal dev server
local MinIO/S3-compatible storage or dev R2
science Docker environment
```

Scientific pipeline should also run locally on a small test dataset without Modal.

Provider adapters determine:

```
LOCAL
MODAL
```

mode.

---

148. Staging

Staging must use the same architectural shape as production:

```
Next.js
Temporal
orchestrator
Modal
R2
Postgres
Polar sandbox
```

The purpose is catching integration failures that local mocks hide.

---

149. Production Release Gates

Before production:

```
auth penetration review passed

cross-tenant automated tests passed

science benchmark passed

load tests passed

backup restore tested

deletion workflow tested

Polar reconciliation tested

workflow crash/recovery tested

Modal failure tested

R2 upload interruption tested

run cancellation tested

quota enforcement tested

audit log tested
```

```
incident runbooks written

Terms/Privacy reviewed
```

---

150. Minimum Load Tests

Simulate:

```
concurrent dashboard users

simultaneous large upload initialization

multiple active uploads

hundreds of polling/status requests

burst of workflow submissions

compute completion burst

Polar webhook burst
```

Do not attempt to load-test actual giant biological datasets exclusively through synthetic HTTP tests.

Scientific benchmarking is separate.

---

151. Data Size Testing

Benchmarks:

```
10k cells
50k cells
100k cells
500k cells
1M cells
```

Measure:

```
import time
peak RAM
CPU time
GPU time
artifact size
```

```
visualization responsiveness
cost
```

Results determine plan limits.

Do not invent commercial file/cell limits before benchmarking.

---

152. Performance Budgets

Application performance targets:

```
dashboard metadata response:
fast enough for interactive B2B usage

long analyses:
fully asynchronous

large uploads:
resumable

result visualizations:
progressively loaded
```

No UI request should wait synchronously for a scientific pipeline.

---

153. Operational Data vs Scientific Data

Separate conceptually:

Operational data

```
users
projects
permissions
billing
workflow states
comments
```

Stored in Postgres.

Scientific data

```
expression matrices
processed AnnData
embeddings
```

```
large tables
exports
```

Stored primarily in object storage.

This separation is fundamental.

---

154. Search

V1 search:

```
projects
datasets
experiments
analysis runs
users
```

Postgres search is sufficient.

Do not introduce Elasticsearch/OpenSearch prematurely.

---

155. Reports

Scientific report generation occurs asynchronously.

Report includes:

```
project
dataset
pipeline
QC
clustering
annotations
markers
comparisons
DE
pathways
provenance summary
limitations/warnings
```

Reports are artifacts.

They never become a separate source of truth.

---

156. Billing Failure Behaviour

If subscription enters payment recovery:

Do not immediately delete scientific data.

Policy:

```
ACTIVE
↓
PAST_DUE
↓
grace period
↓
restricted
```

Restricted state may:

```
allow viewing/export
prevent new expensive analyses
```

Data deletion follows retention policy separately.

Polar subscriptions support payment recovery/dunning behavior, but CellWeave controls product access rules.

---

157. Account Suspension

Possible states:

```
ACTIVE
PAST_DUE
SUSPENDED
CLOSING
DELETED
```

Suspension never means immediate irreversible purge.

---

158. Future Enterprise Architecture

Architecture must allow:

```
Enterprise Organization
        ↓
dedicated R2/S3
dedicated Neon/Postgres
dedicated Temporal task queue
dedicated compute
SSO
```

```
custom retention
audit exports
```

But normal customers remain pooled multi-tenant.

---

159. Architecture Decision Records

Maintain `/docs/adr`.

Examples:

```
ADR-001 AnnData canonical representation

ADR-002 R2 storage

ADR-003 Temporal durable workflows

ADR-004 Modal initial compute provider

ADR-005 Polar billing

ADR-006 processed data only

ADR-007 pseudobulk condition DE

ADR-008 no PHI

ADR-009 separate repositories
```

Changing one requires explicit architectural review.

---

160. System Documentation

Maintain:

```
Architecture
Data model
Security model
Scientific methods
API reference
Runbooks
Deployment guide
Incident response
Billing model
```

```
Data retention
Pipeline validation
```

Documentation changes are part of feature Definition of Done.

---

161. Implementation Order

The build should proceed in this dependency order.

Phase 0 — Foundations

Deliver:

```
all repositories
CI
environment strategy
contract generation
coding standards
ADR system
infrastructure foundation
```

Exit criteria:

All repositories deploy placeholder services to development/staging.

Phase 1 — Identity & Tenancy

Deliver:

```
BetterAuth
Google OAuth
organizations
RBAC
project permissions
organization settings
audit framework
```

Exit criteria:

Automated cross-tenant access tests pass.

Phase 2 — Project & Dataset Foundation

Deliver:

```
projects
experiments
```

```
samples
datasets
dataset versions
R2
multipart uploads
upload sessions
```

Exit criteria:

Large interrupted upload can resume and verify.

Phase 3 — Scientific Import System

Deliver:

```
h5ad importer
10x importer
matrix importer
canonicalization
input checksum
metadata wizard
```

Exit criteria:

All golden importer datasets canonicalize to valid AnnData.

Phase 4 — Workflow Infrastructure

Deliver:

```
Temporal Cloud
orchestrator worker
run state machine
Modal adapter
cancellation
retry
reconciliation
```

Exit criteria:

Kill workers midway through jobs and verify workflows recover correctly.

Phase 5 — QC Pipeline

Deliver:

```
QC metrics
doublets
QC plots
QC review gate
filter confirmation
```

Exit criteria:

Human approval signal reliably resumes Temporal workflow.

Phase 6 — Core Analysis

Deliver:

```
normalization
HVG
PCA
neighbors
UMAP
Leiden
marker discovery
```

Exit criteria:

Golden datasets pass scientific regression thresholds.

Phase 7 — Annotation

Deliver:

```
human/mouse annotation
confidence
manual override
annotation history
```

Exit criteria:

Low-confidence data is never silently forced into confident labels.

Phase 8 — Experiment Comparison

Deliver:

```
conditions
biological replicates
comparison builder
pseudobulk DE
```

```
pathway analysis
scientific warnings
```

Exit criteria:

Sample-level comparison safeguards pass test suite.

Phase 9 — Results Experience

Deliver:

```
interactive UMAP
QC explorer
marker tables
DE tables
pathways
artifact downloads
provenance
```

Exit criteria:

Representative large dataset can be explored without downloading the entire H5AD into browser memory.

Phase 10 — Collaboration

Deliver:

```
comments
annotations
activity
notifications
shared projects
```

Phase 11 — Billing

Deliver:

```
Polar checkout
subscription projection
usage ledger
credits
quota enforcement
overage
customer portal
webhook reconciliation
```

Exit criteria:

Duplicate workflow/webhook events cannot create duplicate usage charges.

Phase 12 — Operations

Deliver:

```
ops console
Sentry
OpenTelemetry
alerts
feature flags
admin tools
support correlation IDs
```

Phase 13 — Security & Governance

Deliver:

```
retention
deletion
break-glass
security headers
rate limits
MFA policies
vendor inventory
incident runbooks
backup restoration tests
```

Phase 14 — Production Qualification

Execute:

```
security testing
load testing
scientific benchmark suite
DR exercise
billing reconciliation
tenant isolation review
failure injection
```

Only after all critical gates pass does CellWeave move to general production.

---

162. Production Definition of Done

CellWeave is not "production ready" merely because it is deployed.

Production-ready means:

```
A failed worker does not lose an analysis.

A duplicate request does not duplicate an analysis.

A duplicate billing event does not duplicate a charge.

One tenant cannot access another tenant's data.

Large datasets never traverse Vercel request bodies unnecessarily.

A pipeline update never changes old provenance.

A scientist can reconstruct how a result was produced.

A customer can export their results.

A customer can delete their data.

A failed provider can be detected.

A DB can be restored.

A compute job can be cancelled.

Scientific regressions block deployment.

Operations staff can diagnose failures.

Billing and usage can be reconciled.

Every expensive job has a cost ceiling.
```

---

163. Final Technology Baseline

Application

```
Next.js App Router
React
TypeScript strict mode
```

```
shadcn/ui
Tailwind
Motion
TanStack Query
Axios
Zod
```

Next.js remains the full-stack application/API framework, using the App Router and Route Handlers.

Identity

```
BetterAuth
Google OAuth
Organizations
RBAC
future SAML/OIDC
```

Database

```
Neon PostgreSQL
Drizzle ORM
```

Scientific storage

```
Cloudflare R2
multipart upload
private objects
presigned URLs
```

Durable orchestration

```
Temporal Cloud
TypeScript Temporal Worker
```

Compute

```
Modal
CPU + GPU
```

Scientific engine

```
Python
```

```
AnnData
Scanpy
SciPy
NumPy
pandas
scvi-tools

validated annotation tools
validated differential-expression tools
validated enrichment tooling
```

Billing

```
Polar
subscription
usage
credits
```

Email

```
Resend
```

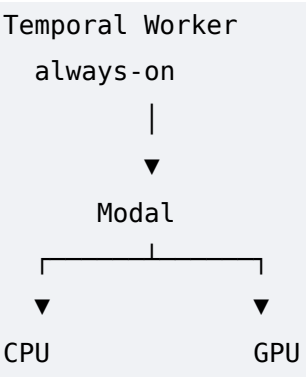
Observability

```
OpenTelemetry
Sentry
structured logging
provider metrics
```

---

164. Deployment Baseline

```
cellweave.com
        |
        ▼
     Vercel
        |
   ┌────┼──────────────┐
   |    |              |
   ▼    ▼              ▼
 Neon   R2        Temporal Cloud
                        |
                        ▼
```



Temporal Worker initially runs on a lightweight always-on container/background-worker platform.

It can later move to:

```
AWS ECS/Fargate
```

without changing application architecture.

---

165. Architecture Rule That Must Never Be Broken

The system has four clean boundaries:

```
WEB / BUSINESS
        |
        ▼
ORCHESTRATION
        |
        ▼
SCIENTIFIC COMPUTE
        |
        ▼
SCIENTIFIC ARTIFACTS
```

The web application should not become a scientific compute server.

The science worker should not become a SaaS backend.

Temporal should not become a business database.

Postgres should not become a scientific file store.

R2 should not become a permissions database.

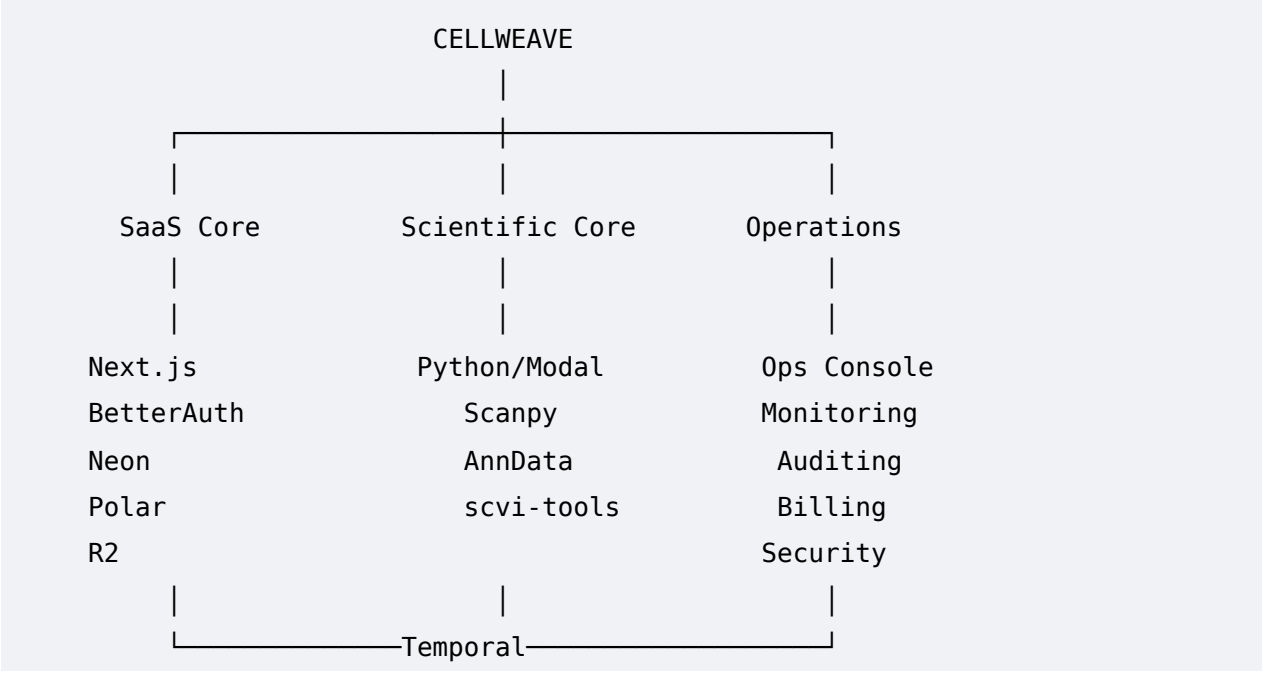
Polar should not become the product's entitlement engine.

Keeping these responsibilities separate is what allows CellWeave to remain maintainable as it grows.

---

166. Product Architecture Summary

CellWeave's final architectural model is:



This is deliberately **not an MVP architecture that must later be thrown away**.

It is a production architecture that can begin small:

```
few organizations
```

and grow through:

```
dozens
hundreds
thousands
```

of organizations by scaling the existing boundaries rather than rewriting the entire platform.

---

167. Final Product Definition

CellWeave is a production-grade, multi-tenant B2B scientific SaaS that enables small and mid-sized biotech and CRO teams to import processed single-cell RNA-seq datasets, perform validated downstream analysis, review and compare experiments collaboratively, and preserve every result with complete computational provenance and reproducibility.

Its primary differentiators are:

```
Vendor-neutral ingestion

Reproducibility-first execution
```

```
Experiment-level comparison

Scientist ↔ bioinformatician collaboration

Production-grade workflow durability

Controlled scientific methodology

Future evidence-backed interpretation
```

The platform shall remain:

```
Research Use Only

non-diagnostic

processed-data-first

human-reviewable

scientifically reproducible

multi-tenant secure

compute-provider portable

billing-provider isolated

scalable without premature infrastructure complexity
```

This document is the architectural source of truth for CellWeave v1.